Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

«Информационная безопасность»

Лабораторная работа № 1

Разработка защищенного REST API с интеграцией в CI/CD

Обучающийся:
Александров Александр Александрович, P3406

Практик:
Маркина Татьяна Анатольевна

2026

Содержание

1 Цель и постановка задачи . . . 3
2 Выбор технологий . . . 3
3 Архитектура приложения . . . 4
3.1 Модель данных . . . 4
4 Реализация API . . . 4
4.1 Аутентификация: POST /auth/login . . . 4
4.2 Получение данных: GET /api/data . . . 5
4.3 Создание данных: POST /api/data . . . 5
5 Реализованные меры защиты . . . 5
5.1 Защита от SQL-инъекций . . . 5
5.2 Защита от XSS . . . 5
5.3 Защита аутентификации . . . 6
5.4 Broken Access Control и дополнительные меры . . . 6
6 Тестирование . . . 6
7 CI/CD и анализ безопасности . . . 7
7.1 SAST: SpotBugs . . . 7
7.2 SCA: OWASP Dependency-Check . . . 8
7.3 Устранение CVE и проверка версий . . . 10
7.4 Secret scanning и DAST . . . 11
8 Заключение . . . 11
9 Источники . . . 12

1 Цель и постановка задачи

Цель работы - получить практический опыт разработки безопасного backend-приложения и встроить автоматическую проверку безопасности в процесс непрерывной интеграции. В работе требуется учесть риски OWASP Top 10, реализовать JWT-аутентификацию, исключить хранение паролей в открытом виде, предотвратить SQL-инъекции и XSS, а также настроить тестирование, контроль покрытия, SAST, SCA, поиск секретов и DAST.

В рамках работы необходимо реализовать не менее трех методов REST API:

1. `POST /auth/login` - проверка логина и пароля, выдача JWT;
2. `GET /api/data` - чтение данных только после аутентификации;
3. самостоятельно выбранный метод - в проекте это `POST /api/data`, создающий новую запись от имени текущего пользователя.

Для автоматической проверки должны выполняться тесты, статический анализ исходного кода или байткода и аудит сторонних зависимостей. Проверки должны запускаться на каждый push и pull request и блокировать интеграцию при нарушении заданных порогов.

2 Выбор технологий

Проект реализован на Java 21 и Spring Boot 4.0.8. Выбранная версия Java является LTS, а Spring Boot предоставляет готовую интеграцию веб-слоя, безопасности и JPA. Переход с Spring Boot 3.5.16 выполнен после завершения OSS-поддержки Spring Framework 6.2 в июне 2026 года; новая платформа использует Spring Framework 7.0.9.

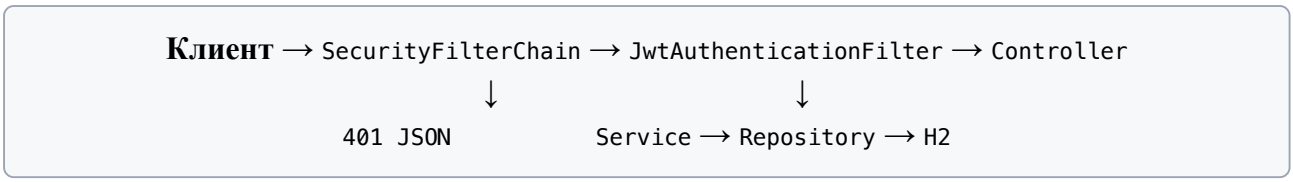
Компонент	**Назначение**
Spring Web	REST-контроллеры, JSON-сериализация и обработка HTTP-запросов.
Spring Security	Проверка учётных данных, BCrypt, фильтрация JWT и контроль доступа.
Spring Data JPA	Работа с базой через ORM и параметризованные запросы.
H2	Файловая база данных для воспроизводимого учебного запуска.
JJWT 0.13.0	Формирование и криптографическая проверка JWT.
JUnit 5 + MockMvc	Интеграционные тесты API без внешнего HTTP-сервера.
JaCoCo	Измерение покрытия и блокирующий порог 70% строк.
SpotBugs	SAST: статический анализ скомпилированного Java-байткода.
OWASP Dependency-Check	SCA: сопоставление зависимостей с базами известных CVE.
Gitleaks	Поиск секретов во всей истории Git.
ZAP Baseline	Пассивный DAST запущенного приложения.
GitHub Actions	Автоматический запуск проверок и публикация artifacts.

Сборка выполняется Maven Wrapper, поэтому на рабочей станции достаточно JDK 21. Все версии библиотек и плагинов закреплены в `pom.xml`, что делает сборку воспроизводимой.

3 Архитектура приложения

Приложение имеет слоистую структуру:

1. `controller` принимает HTTP-запросы и возвращает DTO;
2. `service` содержит сценарии аутентификации и работы с данными;
3. `repository` изолирует доступ к базе данных;
4. `model` содержит JPA-сущности пользователя и записи;
5. `security` реализует создание и проверку JWT;
6. `config` задаёт правила Spring Security и начальные данные;
7. `error` формирует безопасные JSON-ответы об ошибках.



Листинг 1. Упрощённая схема обработки защищённого запроса

Для каждого запроса к `/api/**` цепочка фильтров извлекает Bearer-токен. При корректной подписи, издателе и сроке действия в `SecurityContext` помещается аутентифицированный пользователь. Только после этого запрос передаётся контроллеру.

3.1 Модель данных

Таблица `app_users` содержит идентификатор, уникальное имя пользователя и BCrypt-хеш. Поле с открытым паролем в сущности отсутствует. Таблица `data_items` содержит заголовок, текст, владельца и время создания. Ограничения длины заданы одновременно в DTO и схеме базы.

Учебная учётная запись создаётся при первом старте. Пароль поступает из `APP_DEFAULT_PASSWORD`, хешируется и только после этого сохраняется в H2. Секрет JWT поступает из `JWT_SECRET`; он не включён в репозиторий.

4 Реализация API

4.1 Аутентификация: POST /auth/login

Метод принимает JSON с полями `username` и `password`. Bean Validation запрещает пустые и чрезмерно длинные значения. `AuthenticationManager` передаёт данные в `DaoAuthenticationProvider`, который получает хеш пользователя через JPA и сравнивает его с введённым паролем с помощью BCrypt.

Пример запроса:

```
curl -X POST http://localhost:8080/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"student","password":"StrongPassword123!"}'
```

Успешный ответ:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "tokenType": "Bearer",
  "expiresInSeconds": 900
}
```

При неверной паре логин-пароль возвращается статус `401 Unauthorized` с нейтральным сообщением. Оно не позволяет определить, существовал ли пользователь.

4.2 Получение данных: GET /api/data

Метод доступен только с заголовком `Authorization: Bearer <token>`. Репозиторий возвращает записи в порядке убывания времени создания. Перед сериализацией все пользовательские строки экранируются.

```
curl http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN"
```

Запрос без токена получает `401`:

```
{
  "status": 401,
  "error": "Authentication is required",
  "details": {}
}
```

4.3 Создание данных: POST /api/data

Третий метод создаёт запись от имени аутентифицированного пользователя. Имя владельца берётся из `Authentication`, а не из тела запроса, поэтому клиент не может подменить автора.

```
curl -X POST http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"title":"Новая запись","content":"Содержимое"}'
```

При успехе возвращается `201 Created`. Заголовок ограничен 100 символами, содержимое - 500 символами.

5 Реализованные меры защиты

5.1 Защита от SQL-инъекций

Приложение не формирует SQL путём конкатенации строк. Для доступа к данным используется Spring Data JPA:

```
Optional<AppUser> findByUsername(String username);
List<DataItem> findAllByOrderByCreatedAtDesc();
```

Hibernate связывает значения с параметрами подготовленного выражения. Строка пользователя не становится частью синтаксиса SQL. Отдельный интеграционный тест передаёт имя пользователя `' OR '1'='1` и ожидает `401`, что подтверждает невозможность обхода входа классической SQLi-нагрузкой.

5.2 Защита от XSS

REST API возвращает JSON, но текст может позднее попасть в HTML-интерфейс. Поэтому поля `title`, `content` и `owner` централизованно экранируются при создании `DataResponse`:

```
HtmlUtils.htmlEscape(item.getTitle());
HtmlUtils.htmlEscape(item.getContent());
HtmlUtils.htmlEscape(item.getOwner());
```

Тег `<script>` преобразуется в `<script>`. Дополнительно Spring Security задаёт `X-Content-Type-Options: nosniff`, запрещает встраивание во frame и возвращает CSP `default-src 'none'; frame-ancestors 'none'`.

5.3 Защита аутентификации

Пароль кодируется `BCryptPasswordEncoder(12)`. BCrypt автоматически генерирует уникальную соль и использует регулируемую стоимость. Даже одинаковые пароли имеют разные хеши.

JWT создаётся после успешной проверки учётных данных. В него включены:

1. `sub` - имя пользователя;
2. `iss` - идентификатор сервера `secure-rest-api`;
3. `iat` - время выдачи;
4. `exp` - время окончания действия через 15 минут.

Подпись формируется HMAC-SHA-256 с секретом не менее 256 бит. `JwtAuthenticationFilter` проверяет подпись, `iss`, `exp` и наличие пользователя. Сервер работает без HTTP-сессий (`STATELESS`). Повреждённый или просроченный токен приводит к `401`.

5.4 Broken Access Control и дополнительные меры

Все методы, кроме `/auth/login` и системного `/error`, требуют аутентификацию. Имя владельца записи определяется сервером. Ошибки не раскрывают stack trace. Входные DTO имеют ограничения размера. Секреты, база H2, логи и сборочные файлы исключены через `.gitignore`.

CSRF отключён осознанно: приложение не использует cookie для аутентификации, а Bearer-токен передаётся клиентом в заголовке Authorization. Если в будущем токен будет храниться в cookie, защиту CSRF потребуется включить. Для защиты цепочки сборки Maven Wrapper проверяет SHA-256 архива Maven, а `setup-java` проверяет подпись дистрибутива JDK в CI.

6 Тестирование

Интеграционные тесты запускают полный Spring Context и отправляют запросы через MockMvc. Используется отдельная H2 in-memory база и тестовый секрет JWT.

№	Проверка	Ожидаемый результат
1	Корректный логин и пароль	`200`, JWT, срок 900 секунд
2	GET без Authorization	`401 Unauthorized`
3	GET с действующим JWT	`200`, массив данных
4	SQLi в поле username	`401`, вход не выполнен
5	POST с тегом `<script>`	`201`, HTML экранирован
6	Значение пароля в БД	BCrypt-хеш, не открытый пароль
7	Повреждённый Bearer-токен	`401`, токен отклонён
8	Заголовок длиннее 100 символов	`400`, ошибка валидации

Результат локального запуска:

```
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

JaCoCo включён в фазу `verify`. Сборка блокируется при покрытии менее 70% строк. После миграции на Spring Boot 4.0.8 измерено 98,30%: выполнено 173 из 176 строк. Порог выбран ниже текущего результата, чтобы предотвращать существенную деградацию и оставлять возможность постепенно повышать требование.

Дополнительно приложение проверено вручную последовательностью `curl`: получен JWT, доступ без токена запрещён, с токеном разрешён, новая запись создаётся со статусом 201.

7 CI/CD и анализ безопасности

Workflow `.github/workflows/ci.yml` запускается на события `push` и `pull_request`. Разрешения workflow ограничены чтением содержимого репозитория. Секрет NVD поступает через GitHub Secrets.

Текущий workflow содержит четыре задания: `Secret scanning (Gitleaks)`, `Tests and SAST (SpotBugs)`, `SCA (OWASP Dependency-Check)` и `DAST (ZAP baseline)`. Первые три могут выполняться параллельно. DAST начинается после тестов и использует именно проверенный JAR, опубликованный предыдущим заданием.

Следующий снимок относится к исходной версии pipeline до усиления: тогда существовали только два задания. Он сохранён как фактическое подтверждение выполненного запуска, а не как изображение текущей конфигурации.

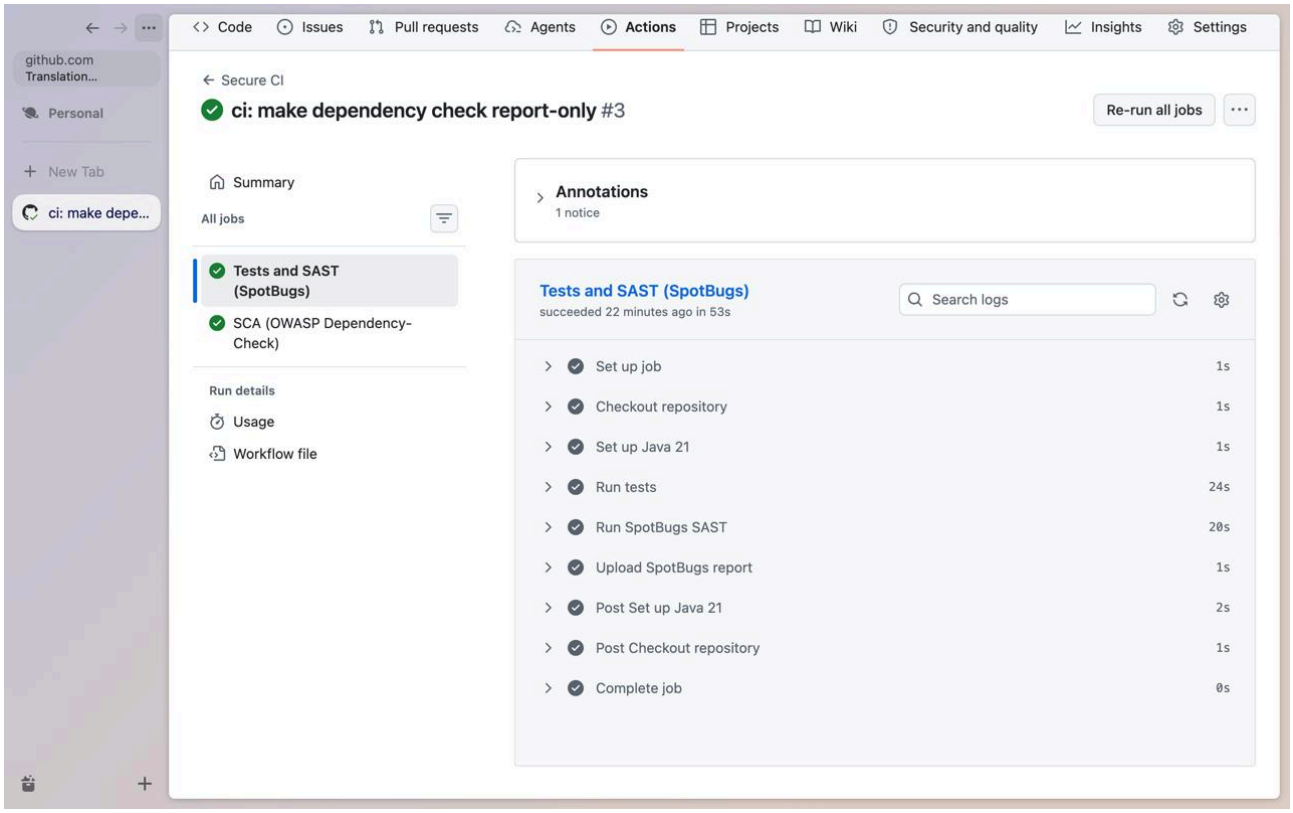


Рис. 1. Исторический запуск Secure CI до добавления coverage, secret scanning и DAST

7.1 SAST: SpotBugs

Задание `Tests and SAST (SpotBugs)` выполняет сборку, тесты, `spotbugs:spotbugs` и `spotbugs:check`. Порог `Medium` делает замечания среднего и высокого уровня блокирующими. HTML/XML-отчёты публикуются как GitHub Actions artifacts даже при ошибке.

Первый прогон обнаружил четыре предупреждения о возможной передаче изменяемых объектов наружу. Реализация была исправлена защитным копированием `Map`; при переходе на Jackson 3 для формирования ответов используется неизменяемый `ObjectWriter`. Повторный локальный запуск после миграции завершился результатом `BugInstance size is 0`, `Error size is 0`.

Фактический лог GitHub Actions подтверждает запуск `spotbugs:spotbugs` и блокирующего `spotbugs:check`. Проверка завершилась сообщениями `BugInstance size is 0`, `Error size is 0`, `No errors/warnings found` и итогом `BUILD SUCCESS`.



Рис. 2. Успешный запуск SpotBugs в историческом CI-прогоне

7.2 SCA: OWASP Dependency-Check

Отдельное задание `SCA (OWASP Dependency-Check)` анализирует прямые и транзитивные зависимости. После обновления платформы порог возвращён к `failBuildOnCVSS=7`: уязвимость уровня High или Critical делает job неуспешным. HTML и JSON создаются и публикуются даже при падении шага, поэтому блокировка не скрывает детали находки.

База NVD кэшируется между запусками; для полного обновления обязателен repository secret `NVD_API_KEY`. Если секрет отсутствует, workflow завершается с понятной ошибкой, а неполная база не выдаётся за успешный аудит. Для сетевых обращений заданы тайм-ауты подключения 30 секунд и чтения 120 секунд. HTML и JSON публикуются как artifacts даже при ошибке шага сканирования.

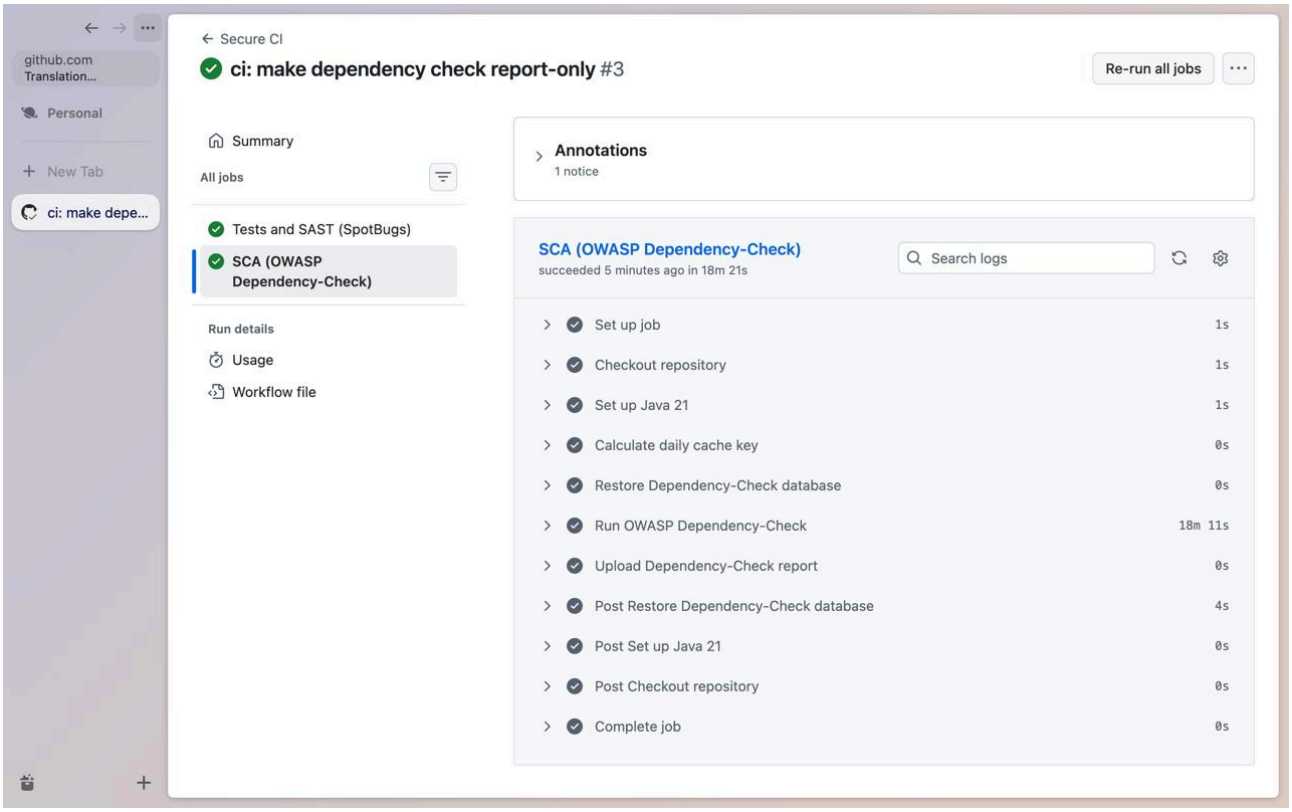


Рис. 3. Историческая структура SCA-job до включения блокирующего порога

Первый SCA-прогон выявил CVE-2026-34477 и CVE-2026-34479 в транзитивной зависимости `log4j-api 2.24.3`, управляемой Spring Boot. В соответствии с рекомендацией Apache Logging Services Log4j BOM обновлён до 2.26.1; после обновления эти две записи для Log4j исчезли из отчёта.

Затем обновлённая база NVD обнаружила 42 CVE в пяти транзитивных компонентах: `jackson-databind 2.21.4`, `spring-core 6.2.19`, `spring-data-jpa 3.5.13`, `spring-security-core 6.5.11` и `tomcat-embed-core 10.1.55`. Корневой причиной стала устаревшая управляемая платформа Spring Boot 3.5.16, а не прямое объявление этих версий в проекте.

На историческом снимке ниже одновременно видны предупреждение об уязвимостях и `BUILD SUCCESS`. Причиной был прежний отчётный порог `failBuildOnCVSS=11`. Следовательно, зелёный статус означал только технически успешное формирование отчёта и не доказывал отсутствие CVE. Именно поэтому порог 11 нельзя оставлять постоянной политикой.

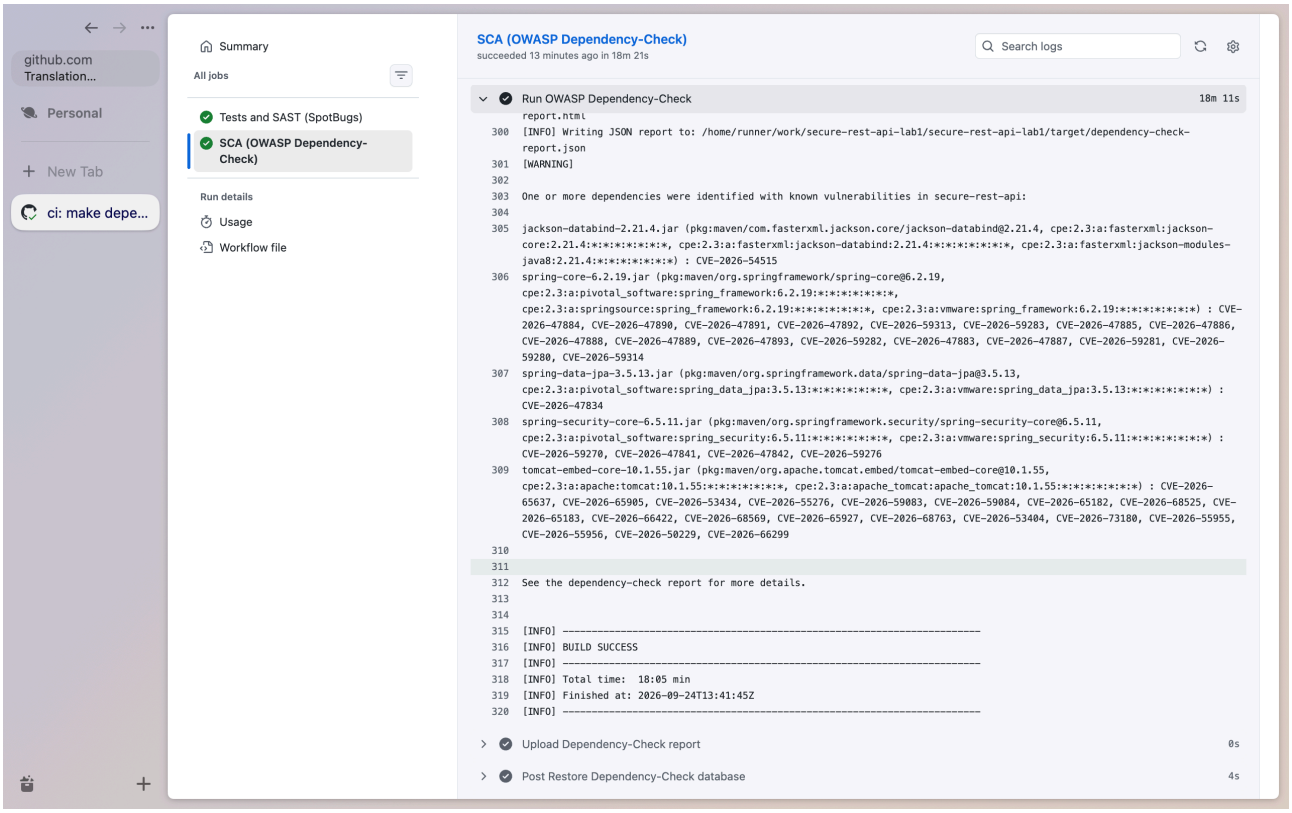


Рис. 4. Историческая находка CVE при прежнем неблокирующем пороге 11

7.3 Устранение CVE и проверка версий

Проект мигрирован на Spring Boot 4.0.8. Эта версия переводит приложение на поддерживаемую линию Spring Framework 7.0.x и одновременно обновляет Jackson и Tomcat. Для Boot 4 стартер `spring-boot-starter-web` заменён на `spring-boot-starter-webmvc`, основной JSON-код переведён с пакетов `com.fasterxml.jackson.databind` на Jackson 3 `tools.jackson.databind`, а тестовый импорт `AutoConfigureMockMvc` обновлён в соответствии с модульной структурой Boot 4.

Компонент	**До**	**После**	**Результат**
Spring Framework	`6.2.19`	`7.0.9`	переход на 7.0.x
Spring Security	`6.5.11`	`7.0.7`	обновлён
Spring Data JPA	`3.5.13`	`4.0.7`	обновлён
Tomcat Embed Core	`10.1.55`	`11.0.24`	обновлён
Jackson Databind 2	`2.21.4`	`2.21.5`	исправлен для JJWT
Jackson Databind 3	—	`3.1.5`	основной JSON-стек

Перегенерированный 18 октября 2026 года локальный HTML/JSON-отчёт проверил 50 зависимостей и показал 0 уязвимых зависимостей и 0 CVE. Для воспроизводимости важно указать ограничение: сканирование выполнялось без сетевого обновления на локальном кэше NVD

от 23 сентября 2026 года. Поэтому окончательный контроль выполняет CI с обязательным `NVD_API_KEY` и свежим обновлением базы.

Файл `dependency-check-suppressions.xml` подключён, но активных правил в нём нет: после обновления подавлять нечего. Если в будущем останется условно неприменимая CVE, правило должно точно связывать пакет и CVE, содержать доказательство недостижимости, владельца или задачу и дату окончания `until`. По истечении срока Dependency-Check снова покажет находку. Suppression не заменяет обновление уязвимой библиотеки.

7.4 Secret scanning и DAST

Checkout с `fetch-depth: 0` делает Gitleaks доступной полную историю Git; action проверяет диапазон коммитов, относящийся к событию `push` или `pull_request`. Поэтому ранее закоммиченный секрет остаётся доступным для анализа, а найденная утечка блокирует отдельный job. Реальные значения JWT, пароля пользователя и пароля БД по-прежнему передаются через переменные окружения.

DAST-job получает JAR, уже прошедший тесты, создаёт одноразовые значения секретов, использует H2 in-memory и запускает приложение на runner. ZAP Baseline выполняет пассивное сканирование. Отсутствие CSP, anti-clickjacking или `X-Content-Type-Options` помечено как `FAIL`; прочие предупреждения сохраняются в artifact. Используется закреплённый commit action версии 0.15.0, что снижает риск неожиданного изменения стороннего CI-кода.

Baseline охватывает неаутентифицированную поверхность и заголовки ответа, но не заменяет полноценный аутентифицированный DAST. Активный scan следует добавлять после публикации OpenAPI-описания и подготовки изолированного окружения, поскольку он намеренно отправляет атакующие запросы.

8 Заключение

В ходе работы создан защищённый REST API на Java 21 и Spring Boot 4.0.8. Реализованы три метода, JWT-аутентификация, BCrypt-хеширование, параметризованный доступ к данным через JPA и экранирование пользовательского содержимого. Защищённые маршруты возвращают 401 без действующего токена.

Настроен GitHub Actions pipeline, запускающий восемь интеграционных тестов, JaCoCo gate, SpotBugs, Dependency-Check, Gitleaks и ZAP Baseline на каждый push и pull request. Миграция устранила уязвимые версии Spring 6.2, Jackson 2.21.4 и Tomcat 10.1.55; порог SCA возвращён к блокирующему значению CVSS 7. Локально подтверждены успешные тесты, покрытие строк 98,30%, отсутствие замечаний SpotBugs и 0 CVE в перегенерированном отчёте на имеющемся кэше NVD. Окончательным результатом SCA считается CI-artifact после обновления базы с `NVD_API_KEY`.

9 Источники

1. OWASP Top 10. https://owasp.org/www-project-top-ten/.
2. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/.
3. JSON Web Token Introduction. https://www.jwt.io/introduction.
4. Spring Security Reference. https://docs.spring.io/spring-security/reference/.
5. GitHub Actions Documentation. https://docs.github.com/actions.
6. SpotBugs Maven Plugin. https://spotbugs.github.io/spotbugs-maven-plugin/.
7. OWASP Dependency-Check. https://dependency-check.github.io/DependencyCheck/.
8. Apache Logging Services Security. https://logging.apache.org/security.html.
9. Spring Framework Versions. https://github.com/spring-projects/spring-framework/wiki/Spring-Framework-Versions.
10. Spring Boot 4.0 Migration Guide. https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide.
11. JaCoCo Maven Plugin. https://www.jacoco.org/jacoco/trunk/doc/maven.html.
12. Gitleaks Action. https://github.com/gitleaks/gitleaks-action.
13. ZAP Baseline Action. https://github.com/zaproxy/action-baseline.